

The `withoutWrapping()` method can be used on the base class if you want to disable the wrapping of the outermost resource. The method will not remove data keys that are manually added to resource collections created by the user.

To work with pagination, you can pass a paginator instance to the collection method of a specific resource. If you look at the paginated responses, they contain meta and links keys with specific information about the state of the paginator. Laravel ships with several methods to set conditional attributes. You can add an attribute conditionally using the `when()` method. It also accepts a function as an argument which is used to calculate the value of the output. If you have defined several conditional attributes, you can merge them by using `mergeWhen()` method. According to Laravel, you should not use `mergeWhen()` method within arrays that comprise of a combination of numeric and string keys. If the numeric keys are not sorted sequentially, you should avoid the usage of this method. To deal with conditional relationships, you can use `whenLoaded()` method. The `whenPivotLoaded()` method can be used to tackle conditional pivot information. The method accepts the pivot table name and function to verify the pivot data as arguments. Laravel also provides a facility to add meta data such as resource links, which you can include within `toArray()` method. The top level meta-data can be defined by adding a `with()` method directly to your resource class. You can add metadata while defining resources with the help of the `additional()` method, which accepts an array of data.

Serialization

Eloquent provides methods with which you can convert models and relationships to either arrays or JSON. You can convert a model and associated relationships to an array using recursive `toArray()` method. Do you want to convert a model to JSON? You can use `toJson()` method, which converts all attributes and relations to JSON. The encoding options can also be specified. It is possible to hide attributes such as passwords, pin number etc from JSON by adding a `$hidden` property. The `visible` property can be used to define a white-list of attributes that you will include in the array and JSON structure. If there are hidden attributes, you can make them visible by using the `makeVisible()` method. The `makeHidden()` method can be employed if you hide visible attributes. The `appends` property helps you to integrate values to JSON. By specifying date format in the cast declaration, you can easily perform date related serialization. Laravel provides the con-